

Phase 1 · Step 1.2 — 직교 A* 경로 탐색

Routing3D · 6 방향 직교 이동 · 맨해튼×cell_mm 휴리스틱 · 모듈 astar.py

1. 개요

점유맵(OccupancyMap, 어느 백엔드든) 위에서 시작 셀 → 목표 셀까지의 최단 직교 경로를 A* 알고리즘으로 찾는다. 이동은 6 방향($\pm X, \pm Y, \pm Z$) 직교만 허용하며 대각선은 금지한다.

- Step 1.2 범위: 비용은 균일 이동비용(셀 1 칸 = cell_mm)만 사용.
- 휴리스틱은 맨해튼 거리(셀 수) × cell_mm → 직교 격자에서 admissible & consistent.
- turn penalty·클리어런스·단 분리 등 비용 튜닝은 Step 1.3(cost.py)에서 확장한다.

2. A* 탐색 흐름도

```
open(우선순위 큐): f = g + h 가 작은 것부터 꺼냄
|
▼ pop current
current == goal ? —예→ came_from 역추적으로 경로 복원 → 반환
| 아니오
▼ 6-이웃 nb 마다
is_blocked(nb)? —예→ 건너뛴 (장애물 / 격자 밖)
| 아니오
▼ g_new = g[current] + step_cost
g_new < g[nb]? —예→ g[nb]=g_new, came_from[nb]=current, open 에 push
|
└ open 이 비면 → 경로 없음(None)
```

3. 핵심 알고리즘

- $f(n) = g(n) + h(n)$. g = 시작부터 누적 실비용, h = 목표까지 추정(휴리스틱).
- admissibility: $h \leq$ 실제 최단비용 → A* 는 최적 경로를 보장. 한 칸 비용이 정확히 cell_mm 이고 h = 맨해튼×cell_mm 이므로 과대평가가 없다.
- consistency: 인접 셀로 갈 때 h 감소량 $\leq$ 이동비용 → closed 재방문 불필요.
- tie-break: heapq 항목에 단조 증가 counter 를 넣어 f 동률 시 안정 정렬·비교 오류 방지.
- closed 집합으로 이미 확정된 셀의 낮은 힙 항목(중복 pop)을 무시한다.

4. 자료구조

이름	타입	설명
open_heap	heapq[(f,counter,cell)]	f 최소 힙. counter 로 안정 tie-break.
g	dict[cell→float]	셀별 최소 누적 비용(mm). 미방문은 $+\infty$ 취급.
came_from	dict[cell→cell]	경로 복원용 직전 셀 맵.
closed	set[cell]	확정(expand)된 셀. 중복 pop 무시.
AStarResult	dataclass	탐색 결과 묶음(아래 표).

AStarResult 필드

필드	설명
success	경로를 찾았는지 여부.
path	셀 리스트 [start..goal]. 실패 시 None.
length_mm	경로 기하 길이(mm) = (셀 수 - 1) × cell_mm.
turns	방향 전환(직각 회전) 횟수.
expanded_nodes	확장한 노드(상태) 수 = 탐색 비용 지표.
visited	확장된 모든 셀(collect_visited=True 일 때). 시각화용.
elapsed_ms	탐색 소요 시간(ms).
cost_mm	총 비용(mm). 균일 A*에서는 length_mm 과 동일.

5. 주요 함수

함수	역할
astar(occ, start, goal, *, step_cost, collect_visited, max_expansions)	균일 비용 직교 A*. 상태 = 셀.
astar_weighted(occ, start, goal, params, ...)	비용함수 A*(Step 1.3). 상태 = (셀, 진입방향). cost.py 참조.
astar_world(occ, start_mm, goal_mm, **kw)	월드 좌표(mm)를 셀로 변환 후 astar 호출하는 편의 함수.
manhattan(a, b)	두 셀의 맨해튼 거리(셀 수). 휴리스틱 기반.
count_turns(path)	경로의 방향 전환 횟수 계산.
_reconstruct(came_from, goal)	came_from 역추적 → 정방향 경로 리스트.

6. 주요 변수 / 파라미터

이름	기본값	설명
----	-----	----

step_cost	None→cell_mm	셀 1 칸 이동 비용(mm).
collect_visited	False	True 면 확장 셀을 모두 모아 시각화/디버그에 사용.
max_expansions	None	확장 셀 수 상한(폭주 방지). None=무제한.
sc	cell_mm	내부: 실제 step_cost.
expanded	0	내부: 누적 확장 노드 수.

7. 실패(success=False) 조건

- start 또는 goal 이 점유 / 격자 밖.
- 목표까지 경로가 없음(open 소진).
- max_expansions 초과.

8. 실행 명령어

```
# DB 영역에서 start→goal(mm) 경로 탐색 + 지표 출력
.\.venv\Scripts\python.exe -m routing3d_py.astar ^
  --region 195000 8000 14000 205000 12000 16000 --cell-mm 50 ^
  --start 196000 9000 15600 --goal 204000 11000 15600

# 경로+점유맵 3D 렌더(스크린샷)
.\.venv\Scripts\python.exe -m routing3d_py.astar ^
  --region 195000 8000 14000 205000 12000 16000 ^
  --start 196000 9000 15600 --goal 204000 11000 15600 ^
  --screenshot python_experiments/out/route.png

# 단위 테스트
.\.venv\Scripts\python.exe -m pytest python_experiments/tests/test_astar.py -v
```